JDK, JRE and JVM

11 min read · Updated 28 Jul 2026 · Topic 1 of 65

Bookmark · Mark done

Why it matters

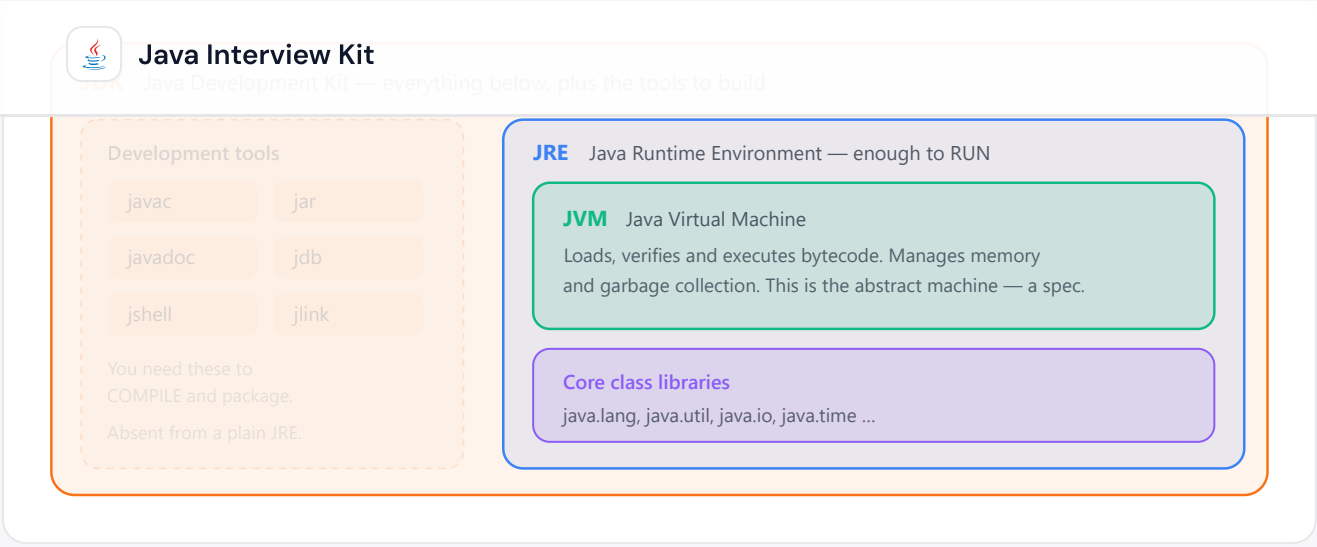
This is the question that opens a startling number of Java interviews, and it is not really a warm-up. It is a filter. Almost every candidate can recite "JDK is for development, JRE is for running, JVM runs the bytecode" — that sentence is on the first page of every tutorial ever written. What separates a strong answer is understanding *what kind of thing* each of the three actually is: one is a specification, one is a bundle of libraries, one is a toolchain. Candidates who see them as three sizes of the same product get quietly marked down.

It matters practically too. "It compiles on my machine but the server says `UnsupportedClassVersionError`", "the Docker image is 400 MB and I only need to run a jar", "which JDK do I actually install — Oracle, Temurin, Corretto?" — every one of those is the same knowledge in working clothes. Getting this topic right makes the next thirteen chapters make sense, because everything from garbage collection to Spring's classpath scanning happens *inside* the boundary this topic draws.

The concept, simply

Plainly: they are three nested things, each containing the one below it.

- **JVM** — the machine that executes your compiled code. It is the innermost layer, and it is the reason Java is portable.
- **JRE** — the JVM *plus* the standard library your code calls (`java.lang`, `java.util`, and the rest). Enough to *run* a Java program, and nothing more.
- **JDK** — the JRE *plus* the tools that turn source code into something runnable: the compiler `javac`, the packager `jar`, the debugger `jdb`, and friends.



Strict containment: the JDK is a superset of the JRE, which is a superset of the JVM. Install the JDK and you already have all three.

More precisely — and this is the sentence that earns marks — **the JVM is a *specification***. Oracle publishes a document, the *Java Virtual Machine Specification*, that defines an abstract computing machine: its instruction set, its memory areas, the class file format it accepts. Anyone may implement it. HotSpot (the one you almost certainly run) is Oracle's implementation; Eclipse OpenJ9 is IBM's; there have been many others. When you say "the JVM", you are usually naming the spec but running one particular implementation of it.

How it works

Work outward from the middle.

The JVM is handed a `.class` file full of bytecode and is responsible for four jobs: *loading* classes into memory, *verifying* that the bytecode is safe and well-formed, *executing* it, and *managing memory* — including garbage collection. It is a stack-based machine: its instructions push and pop from an operand stack rather than naming CPU registers, which is precisely what makes the same bytecode work on x86 and ARM without change. Chapter 9 opens this box up properly.

The JRE adds what the JVM alone cannot supply: the class libraries. Your `String`, your `ArrayList`, your `HashMap` — none of that is built into the JVM instruction set. They are ordinary Java classes shipped with the runtime. The JRE is therefore "the JVM plus everything `java.*`", packaged so that a machine that will only ever *run* Java programs needs nothing else.

The JDK adds the toolchain. The important members:

Tool	What it does
`javac`	Compiles `.java` source into `.class` bytecode
`java`	Launches the JVM and runs a class or jar
`jar`	Packages many `.class` files into one archive
`javadoc`	Generates API documentation from comments
`jdb`	Command-line debugger
`jshell`	Interactive REPL (Java 9+) — excellent for testing a snippet
`jlink`	Builds a trimmed custom runtime containing only the modules you use (Java 9+)
`jcmd`, `jstack`, `jmap`	Diagnostics — thread dumps, heap dumps, live JVM inspection

One modern wrinkle worth knowing, because it catches people out: **since Java 11, Oracle no longer ships a standalone JRE download**. The recommended way to produce a runtime-only distribution is now `jlink`, which assembles a custom image containing just the modules your application actually needs — often far smaller than the old JRE. So the JDK/JRE distinction is still conceptually exact, but "download the JRE" is dated advice. If an interviewer asks how you would ship a small runtime today, `jlink` is the answer they are hoping for.

The second thing people conflate is **the JDK versus the vendor**. OpenJDK is the open-source reference codebase. Oracle JDK, Eclipse Temurin, Amazon Corretto, Azul Zulu and Red Hat's build are all builds of essentially that same source, differing in licensing, support window and a few extras — not in the language. "Which JDK should we use?" is a licensing and support question, not a technical one, and saying so demonstrates real-world experience.

Code examples

The distinction is easiest to *feel* on the command line. Compilation needs the JDK; execution needs only the runtime.

BASH Copy

```
# javac ships with the JDK. A runtime-only install does not have it.
javac Hello.java        # Hello.java  →  Hello.class   (source → bytecode)

# java is the JVM launcher — present in every runtime.
java Hello              # loads Hello.class and executes it

# Since Java 11 you can run a SINGLE source file directly. It is still
# compiled first — just in memory, with no .class file left on disk.
java Hello.java
```

Ask the runtime what it is, which is the fastest way to diagnose a version mismatch on a server:

BASH Copy

```
java -version
# openjdk version "17.0.9" 2023-10-17
# OpenJDK Runtime Environment Temurin-17.0.9+9   ← the JRE layer
# OpenJDK 64-Bit Server VM Temurin-17.0.9+9      ← the JVM implementation

javac -version    # if this fails, you have a runtime, not a JDK
# javac 17.0.9
```

A program can inspect its own runtime, which is occasionally useful in diagnostics endpoints:

JAVA Copy

```
public class RuntimeInfo {
    public static void main(String[] args) {
        System.out.println(System.getProperty("java.version"));      // 17
        System.out.println(System.getProperty("java.vm.name"));      // Op
        System.out.println(System.getProperty("java.vendor"));       // Ec
```

```
        System.out.println(Runtime.version().feature());          // 17
        // The JVM's view of the machine and of its own memory budget.
        Runtime rt = Runtime.getRuntime();
        System.out.println(rt.availableProcessors() + " cores");
        System.out.println(rt.maxMemory() / 1024 / 1024 + " MB max heap");
    }
}
```

Under the hood

The most common runtime error in this area is `UnsupportedClassVersionError`, and it is worth understanding rather than memorising, because it explains what "compatibility" means in Java.

Every `.class` file begins with the magic number `0xCAFEBABE` followed by a **major version** — 61 for Java 17, 65 for Java 21. A JVM refuses to load a class whose major version is higher than it supports. Java is therefore **backward compatible but not forward compatible**: a Java 17 runtime happily runs classes compiled for Java 8, but a Java 8 runtime cannot run classes compiled for 17. Compile with a newer JDK than your server runs and you get exactly that error, usually at the worst moment.

The fix is not "downgrade the JDK". It is `--release`, which compiles against the older platform's API as well as emitting the older class format:

BASH Copy

```
# Right: compiles for 11 AND checks you used no API newer than 11.
javac --release 11 Hello.java

# Historic and subtly broken: emits v11 bytecode but still compiles
# against the CURRENT JDK's libraries, so a Java 17-only method call
# compiles fine and then throws NoSuchMethodError at run time on Java 11.
javac -source 11 -target 11 Hello.java
```

The other mechanism worth naming is **bytecode verification**. Before executing a loaded class, the JVM proves properties about it: the operand stack never underflows, every local variable is read with the type it was written with, control flow never jumps outside

`.class` file that reads arbitrary memory. Verification is also why a corrupt jar fails loudly at load time rather than corrupting the heap silently.

Common mistakes

- **"The JDK is a bigger JVM."** They are different kinds of thing. The JVM is a specification (and its implementations); the JDK is a distribution of software you install.
- **Installing a JRE and wondering why `javac` is missing.** Compilation lives only in the JDK. On any dev machine, install the JDK.
- **Thinking every JVM is HotSpot.** HotSpot is the common implementation, not the definition. OpenJ9 has genuinely different GC and startup characteristics.
- **Confusing JDK *version* with JDK *vendor*.** "Java 17" is a version; Temurin and Corretto are vendors of it.
- **Using `-source` / `-target` instead of `--release`.** The former lets you compile calls to APIs that do not exist on the target runtime — a failure that survives compilation and appears in production.
- **Saying the JRE is deprecated.** The standalone Oracle *download* is gone; the runtime layer is not. Say "you build one with `jlink` now" and you sound current instead of confused.

Interview lens

> **Q: "What is the difference between JDK, JRE and JVM?"**
> A: They are nested. The JVM is the abstract machine that loads, verifies and executes bytecode and manages memory — it is a specification, with implementations like HotSpot and OpenJ9. The JRE is the JVM plus the standard class libraries: enough to run a Java program. The JDK is the JRE plus the development toolchain — `javac`, `jar`, `javadoc`, `jshell`, diagnostics. You compile with a JDK; you can run with just a runtime.

A: Part of it, by containment — installing a JDK gives you a JVM. But the JVM is defined by a public specification, so it is not *owned* by any one JDK: several vendors implement it, and that is exactly why the same bytecode runs everywhere.

Q: "Can I run Java without a JDK?"

A: Yes — you need a runtime, not a compiler. Historically that was a JRE download; since Java 11 the modern route is a custom image built with `jlink`, or a vendor's runtime-only distribution. What you cannot do is *compile* without a JDK.

Q: "Why does Java need a virtual machine at all?"

A: It is what buys portability and safety. Compiling to bytecode rather than native code means one artefact runs on any platform with a conforming JVM. And because a JVM sits between the code and the hardware, it can verify bytecode before running it, manage memory automatically, and optimise at run time using information a static compiler never has — like which branch is actually hot.

Q: "What is `UnsupportedClassVersionError` and how do you fix it?"

A: The class file's major version is newer than the running JVM supports — compiled on a newer JDK than the server runs. Fix it by compiling with `--release <target>`, which emits the older class format *and* restricts you to that release's API. Avoid the old `-source`/`-target` pair, which lets newer API calls slip through and fail at run time with `NoSuchMethodError`.

Q: "Which JDK distribution would you choose for a production service?"

A: That is a support and licensing decision, not a language one. Eclipse Temurin or Amazon Corretto are common free, long-supported builds of OpenJDK; Oracle JDK matters when a client requires Oracle support. What I would insist on is pinning an

LTS version — 17 or 21 — and using the same build in CI and production so class-version and GC behaviour match.

Q: "Is Java platform independent? Fully?"

A: The *bytecode* is platform independent; the JVM is not, and cannot be — it is a native binary compiled for each OS and architecture. The usual phrasing is that Java is platform independent but the JVM is platform dependent. And "write once, run anywhere" holds for the language and libraries, not for anything you do outside them: native code via JNI, file path separators or OS-specific commands break it just fine.

Cheat-card

- **JVM ⊂ JRE ⊂ JDK** — strict containment. Install the JDK, get all three.
- **JVM** = specification + implementations (HotSpot, OpenJ9). Loads, verifies, executes, manages memory.
- **JRE** = JVM + core class libraries. Run only.
- **JDK** = JRE + `javac`, `jar`, `javadoc`, `jdb`, `jshell`, `jlink`, diagnostics.
- No standalone Oracle JRE since **Java 11** — build runtimes with `jlink`.
- Bytecode is portable; **the JVM is platform-specific** by necessity.
- **Backward compatible, not forward**: newer JVM runs older classes, never the reverse → `UnsupportedClassVersionError`.
- Cross-compile with `--release`, never `-source`/`-target`.
- Version (17, 21) ≠ vendor (Temurin, Corretto, Oracle).

QUICK CHECK — no marks, just for you.

1. Which statement about the JVM is most accurate?

- ○ The JVM is a specification, with multiple implementations such as HotSpot and OpenJ9

- ◯ The JVM is a single program written by Oracle and shipped only with the Oracle JDK
- ◯ The JVM is another name for the JDK's compiler, javac

Check answer

2. You install a Java runtime on a build server and `javac` is not found. What is the most likely cause?

- ◯ The JVM must be started before javac becomes available
- ◯ You installed a runtime-only distribution; the compiler ships only with the JDK
- ◯ javac was removed from Java in version 11 and replaced by jlink
- ◯ javac only exists on Linux builds of Java

Check answer

3. An application compiled on JDK 17 fails on a Java 11 server with UnsupportedClassVersionError. What is the correct fix?

- ◯ Recompile with -source 11 -target 11
- ◯ Raise the server's -Xmx so the newer classes fit in memory
- ◯ Recompile with --release 11
- ◯ Catch the error at startup and fall back to an older code path

Check answer

Next
How Java Runs: Compilation, Bytecode and the JIT

LEARN MORE

ARTICLES & DOCS

- The Java Virtual Machine Specification, Java SE 17
  docs.oracle.com
- jlink — assemble a custom runtime image
  docs.oracle.com
- Getting started with Java — dev.java
  dev.java

VIDEOS



How Java Works: Understanding JVM, JRE, and JDK | Java Architecture | Java Tutorial For Beginners
Talented Developer

Hand-picked references for this lesson.